Lecture 6: Reinforcement Learning in Finance

Chao ZHANG

March 10, 2026

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

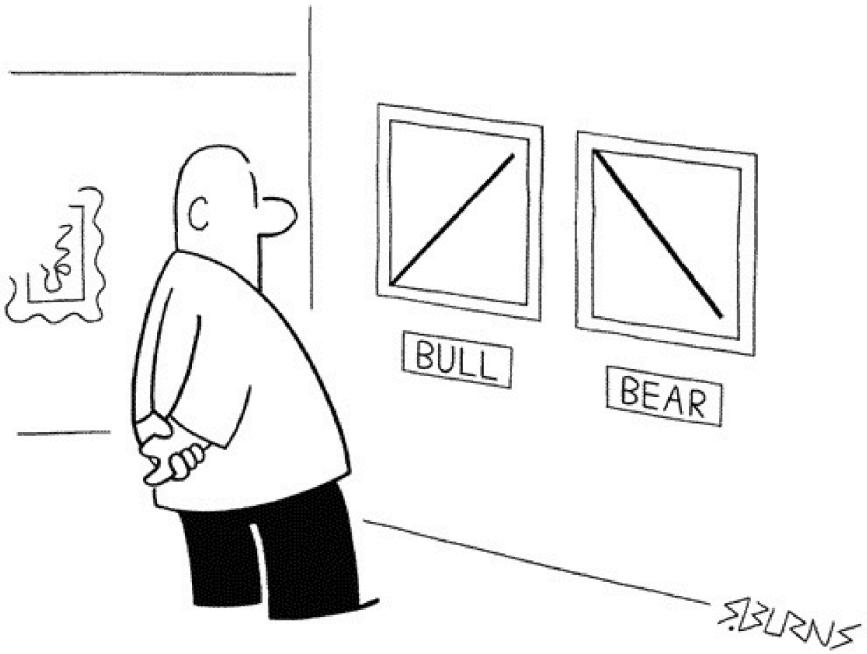
Optimal Liquidation: Sequential Decision under Uncertainty

A Basic Question:

How to optimally liquidate (sell) a portfolio with X shares over a given time period $[0, T]$?

Classic Trade-off

- Liquidating fast is costly: market impact.
- But if one liquidates too slowly...



©2000 Shannon Burns www.shannonburns.com

Optimal Liquidation: Motivation

... the price may go down while we are liquidating...

... and we would have been better executing faster.

$\Longrightarrow$ Need to find an optimal trading schedule to balance market impact and price risk:

- Trading cost (market impact)
- Price risk (stochastic price evolution)

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

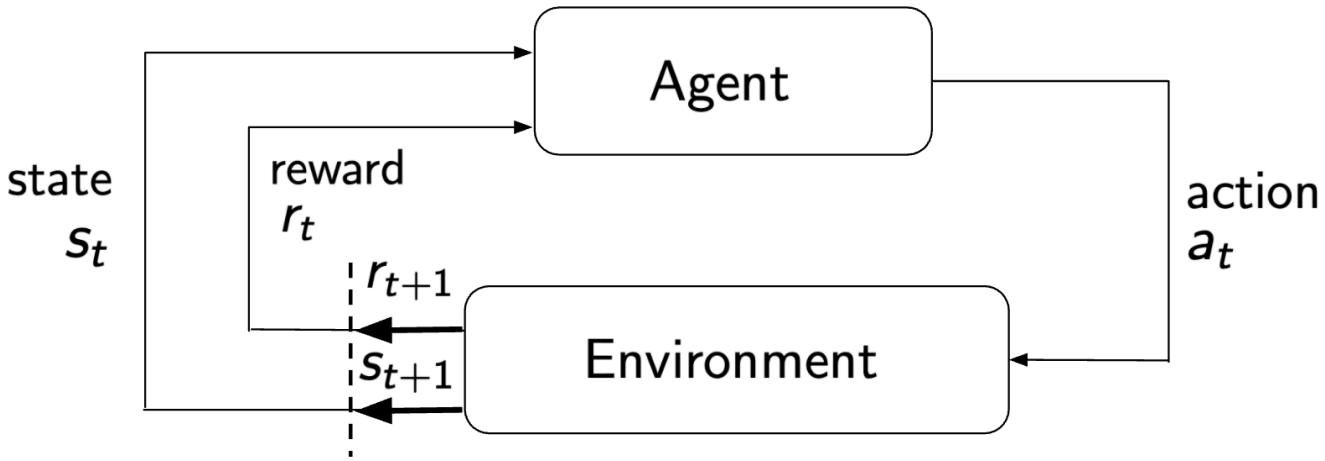
Deep RL

RL in Finance

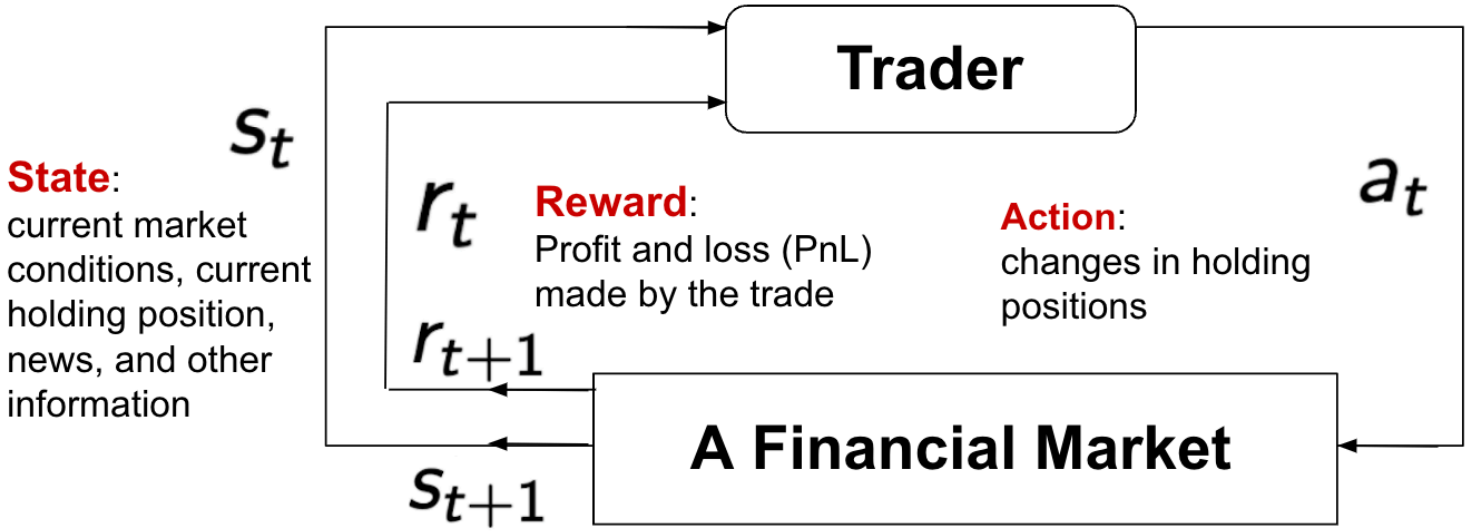
What is Reinforcement Learning?

Reinforcement learning (RL) is a framework for sequential decision making under uncertainty:

- An agent (learner or decision-maker) operates in an environment (possibly unknown to the agent)
- An agent takes actions (a_t) which influences future state in the environment (s_{t+1})
- Success is measured by a reward signal (r_t) (agent may not know the distribution of the signal)
- **Goal** is to select actions to maximize future rewards



Reinforcement Learning in Finance



Reinforcement learning (RL) in the financial context:

- A trader (or investor) operates in an environment (possibly unknown to the trader)
- A trader takes actions (a_t) which influences its future state (think about the market impact) in the market (s_{t+1})
- Success is measured by a reward signal (r_t) (such as return or risk-adjusted return)
- **Goal:** is to select actions to maximize future rewards (profit)

Example: Optimal Execution

Problem Description:

- Optimal execution of large (buying or selling) positions over a given trading period is an important question for all banks, hedge funds, and institutional investors.
- Trade-off between (1) fast trading speed with less profitable execution prices and (2) slow trading speed with exposure to price risk.

Example: Optimal Execution

Problem Description:

- Optimal execution of large (buying or selling) positions over a given trading period is an important question for all banks, hedge funds, and institutional investors.
- Trade-off between (1) fast trading speed with less profitable execution prices and (2) slow trading speed with exposure to price risk.

$\hookrightarrow$ formulate the optimal execution problem in the context of RL:

- **State variable** s: mid-price, current inventory, and other market conditions
- **Action** a: number of shares to liquidate at any timestamp
- **Reward** r: cash inflow, implementation shortfall, or PnL

Main Framework for RL: Markov Decision Process

- A Markov Decision Process (MDP) provides a mathematical framework for modeling sequential decision problems.
- Reinforcement learning studies how an agent can learn optimal behavior in an MDP through interaction with the environment.
- RL is particularly useful when the transition dynamics or reward function are unknown.

- **Markov property (informally):**
  The current state contains all relevant information for the future evolution of the system.

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Markov Decision Process

A MDP is specified by a tuple $(\mathcal{S}, \mathcal{A}, P, R, H)$,

- $\mathcal{S}$ is the set of states (assumed to be finite with size $S = |\mathcal{S}|$)
- $s_1 \in \mathcal{S}$ the starting state
- $\mathcal{A}$ is the set of actions (assumed to be finite with size $A = |\mathcal{A}|$)

Markov Decision Process

A MDP is specified by a tuple $(\mathcal{S}, \mathcal{A}, P, R, H)$,

- $\mathcal{S}$ is the set of states (assumed to be finite with size $S = |\mathcal{S}|$)
- $s_1 \in \mathcal{S}$ the starting state
- $\mathcal{A}$ is the set of actions (assumed to be finite with size $A = |\mathcal{A}|$)
- $P(s' \mid s, a)$ is the transition probability.
- $R(s, a)$ or r_t is the reward signal
- Time Horizon (H)
- The process starts from an initial state s_1 and proceeds in discrete time steps $t = 1, \ldots, H$.
  - At time t, the agent observes state s_t
  - takes action a_t
  - then observes reward r_t and next state s_{t+1}

Markov Property

"The future is independent of the past given the present"

Markov Property

"The future is independent of the past given the present"

The Markov property is formally stated as the following two properties:

- The probability of transitioning to a particular state depends only on current state and action, and not on any other aspect of the history. That is,

$$\mathbb{P}(s_{t+1} = s' | \text{history till time t})$$
$$= \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a) =: P_t(s, a, s')$$

  Here $\sum_{s'} P_t(s, a, s') = 1$.

- The reward distribution depends only on the current state and action. So, that the expected reward at time t is a function of current state and action. A matrix R specifies these rewards.

$$\mathbb{E}[r_t \,|\, \text{history till time t}]$$
$$= \mathbb{E}[r_t \,|\, s_t = s, a_t = a] =: R_t(s, a)$$

Policy: How the Agent Decides

A policy π specifies what action to take at any given state:

- **History-dependent policy** (π_t): Maps the entire history $\mathcal{H}_t = (s_1, a_1, \ldots, s_t)$ to an action a_t.
- **Markovian policy** (π): A mapping from the current state to an action: $\pi : \mathcal{S} \to \mathcal{A}$ (or a distribution $\pi(a|s)$).
- **Optimality**: In an MDP, for any history-dependent policy, there exists a Markovian policy that achieves the same (or better) expected total reward.
  - *Insight: The current state s_t is a sufficient statistic for decision-making.*
- **The RL Goal**: To learn an **optimal policy** π^* that maximizes cumulative rewards when the transition P and reward R are unknown.

Deterministic vs. Randomized Policies

Deterministic Policy: $a = \pi(s)$

A direct mapping from states to actions.

- Financial Example: A simple Mean-Reversion rule.
  - If z-score $< -1 \implies$ Long 1 unit.
  - If z-score $> 1 \implies$ Short 1 unit.

Deterministic vs. Randomized Policies

Deterministic Policy: $a = \pi(s)$

A direct mapping from states to actions.

- Financial Example: A simple Mean-Reversion rule.
  - If z-score $< -1 \implies$ Long 1 unit.
  - If z-score $> 1 \implies$ Short 1 unit.

Randomized Policy: $\pi(a|s) = \mathbb{P}(a_t = a | s_t = s)$

A mapping to a probability distribution over actions $\mathcal{P}(\mathcal{A})$.

- Financial Example: Order Sizing with exploration.
  - Upon signal, sample order size $a \sim \text{Unif}\{1, \ldots, 5\}$.
- **Key Role in RL:** Essential for *exploration* and handling *non-deterministic environments*.

Objective of MDP

Goals. Sequential decision problems involve a trade-off between immediate rewards and future rewards. The objective of an MDP is to choose a policy that maximizes expected cumulative reward.

At a high level, the goal is to maximize expected cumulative reward. Some popular forms are

- (expected) total reward
- (expected) average reward
- (expected) discounted sum of rewards

Finite-horizon MDP

- **Timeline**: Actions are taken for $t = 1, \ldots, H$, where H is the **finite horizon**.
- **Objective**: Maximize the expected total (discounted) rewards in an **episode** of length H.
- For a discount factor $0 \leq \gamma \leq 1$, the goal is to maximize:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=1}^{H} \gamma^{t-1} r_t \mid s_1 \right] \tag{1}$$

  Note: We usually set $\gamma = 1$ when H is small (e.g., high-frequency execution).
- **Example: American Option Pricing**
  - The decision to exercise must happen before or at expiry T.
  - Time-to-maturity is a critical component of the state s_t.

Remark: Episodic MDP

In the RL literature, a finite-horizon MDP is often called an **Episodic MDP**. Each completion of H steps is one **episode** (or a "trajectory").

Infinite horizon MDP

Expected total discounted reward criteria: the goal is to maximize

$$\mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \middle| \; s_1 \right] \tag{2}$$

- This is an asymptotic weighted sum of rewards, where with time the weights decrease by a factor of $\gamma < 1$
- This essentially means that the immediate returns are more valuable than those far in the future
- One of the most popular form – theoretically easier to analyze and makes sense for many practical applications
- Example: long-term portfolio optimization

Infinite horizon MDP

Expected total reward criteria: Maximize

$$\lim_{T\to\infty} \mathbb{E}\left[\sum_{t=1}^{T} r_t \middle| \; s_1 \right] \tag{3}$$

Expected average reward criteria: Maximize

$$\lim_{T\to\infty} \mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T} r_t \middle| \; s_1 \right] \tag{4}$$

- Intuitively, the performance in a few initial rounds does not matter here, what we are looking for is a good asymptotic performance

Optimal Policies: The Search for π^*

- An **optimal policy** π^* is defined as the one that maximizes the expected cumulative gain of the MDP for all states $s \in \mathcal{S}$.
- **Theoretical Guarantee:** For any MDP, there exists at least one deterministic Markovian policy that is optimal.
- Henceforth, we restrict our search to the space of **Markovian policies**.

Optimal Policies: The Search for π^*

- An **optimal policy** π^* is defined as the one that maximizes the expected cumulative gain of the MDP for all states $s \in \mathcal{S}$.
- **Theoretical Guarantee:** For any MDP, there exists at least one deterministic Markovian policy that is optimal.
- Henceforth, we restrict our search to the space of **Markovian policies**.

Setting: Discrete-time, Infinite-horizon, Discounted Reward

To solve for π^*, we introduce two fundamental diagnostic tools:

1. **Value Function** $V^\pi(s)$: "How good is this state?"
2. **Action-Value Function** $Q^\pi(s, a)$: "How good is this action in this state?"

State-Value Function $V^\pi(s)$

The state-value function under policy π is:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \mid S_0 = s \right]$$

- Expected total reward when starting from state s and following policy π
- Measures how good a state is under a given policy.
- Accounts for both immediate and future rewards.
- Helps compare different states for long-term planning.

Action-Value Function $Q^\pi(s, a)$

Knowing the value of a state isn't enough to make a decision!

The action-value function measures how good it is to take action a in state s:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid S_0 = s, A_0 = a \right]$$

The Relationship between Q and V

- **One-step look-ahead:**
  $Q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\pi(s')$
- **Average over actions:** $V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q^\pi(s, a)$

Why is $Q^\pi(s, a)$ Essential?

- $V^\pi(s)$ evaluates the *situation*; $Q^\pi(s, a)$ evaluates the *choices*.
- If we have $Q^*(s, a)$, the optimal policy is simply $\pi^*(s) = \arg\max_a Q^*(s, a)$.

Bellman Equations: The Power of Recursion

Current value is the sum of what you get now and the discounted value of where you land.

State-Value Function $V^\pi(s)$:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \Big[\underbrace{R(s,a)}_{\text{Immediate Reward}} + \gamma \underbrace{\sum_{s'} P(s'|s,a) V^\pi(s')}_{\text{Value of the Next State}} \Big]$$

Action-Value Function $Q^\pi(s,a)$:

$$Q^\pi(s,a) = \underbrace{R(s,a)}_{\text{Immediate}} + \gamma \underbrace{\sum_{s'} P(s'|s,a) \sum_{a'} \pi(a'|s') Q^\pi(s',a')}_{\text{Future (Next Q)}}$$

Intuition for Finance:

- **Decomposition:** Just look one step ahead.
- **Mechanism:** This recursion is the foundation for Dynamic Programming and all RL algorithms.

Example: Computing the Bellman Equation

- **States:** s_1, s_2 **Action:** a **Discount:** $\gamma = 0.9$
- **Rewards:** $R(s_1, a) = 5$, $R(s_2, a) = 2$
- **Transitions:**
  - From s_1: $P(s_1|s_1, a) = 0.8$, $P(s_2|s_1, a) = 0.2$
  - From s_2: $P(s_2|s_2, a) = 1.0$ (Absorbing State)

Example: Computing the Bellman Equation

- **States:** s_1, s_2 **Action:** a **Discount:** $\gamma = 0.9$
- **Rewards:** $R(s_1, a) = 5$, $R(s_2, a) = 2$
- **Transitions:**
  - From s_1: $P(s_1|s_1, a) = 0.8$, $P(s_2|s_1, a) = 0.2$
  - From s_2: $P(s_2|s_2, a) = 1.0$ (Absorbing State)

Bellman Equations:

$$V(s_1) = \underbrace{5}_{\text{Imm.}} + 0.9\Big[\underbrace{0.8\,V(s_1) + 0.2\,V(s_2)}_{\text{Future Value (Expectation)}}\Big]$$

$$V(s_2) = \underbrace{2}_{\text{Imm.}} + \underbrace{0.9\,V(s_2)}_{\text{Future}}$$

Solution:

- $V(s_2) = 2 + 0.9\,V(s_2) \Rightarrow 0.1\,V(s_2) = 2 \Rightarrow \mathbf{V(s_2) = 20}$
- Substitute $V(s_2)$ into $V(s_1)$:
  - $V(s_1) = 5 + 0.9\big[0.8\,V(s_1) + 0.2(20)\big] \Rightarrow \mathbf{V(s_1) \approx 30.71}$

Optimal Value Functions: $V^*(s)$ and $Q^*(s, a)$

Definitions

- The **Optimal State-Value Function** $V^*(s)$:

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

- The **Optimal Action-Value Function** $Q^*(s, a)$:

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

- **Fundamental Relationship:**

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

- $V^*(s)$ is the **maximum potential wealth** achievable from state s.
- $Q^*(s, a)$ is the value of committing to action a now, assuming optimal decisions in all future steps.

Optimal Policy π^*

The optimal policy chooses the action that maximizes future rewards:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

Key Takeaways:

- Always select the action with the highest $Q^*(s, a)$.
- Leads to the best possible long-term outcome.

How to find Q^*: Bellman Optimality Equation

The Self-Consistency of Excellence: The optimal value Q^* must satisfy the condition that tomorrow's value is also optimal.

The Equation for $Q^*(s, a)$

$$Q^*(s,a) = \underbrace{R(s,a)}_{\text{Immediate}} + \gamma \underbrace{\sum_{s'} P(s'|s,a) \max_{a'} Q^*(s',a')}_{\text{Best possible Future}}$$

Why this is powerful:

- **Non-linearity:** Unlike the expectation equation (which is linear), the max operator makes this a **non-linear** system.
- **Fixed Point:** Q^* is the unique solution to this equation.

Note: We no longer need the policy π in the formula because the max *operator implicitly defines the best possible policy.*

Solving for Q^* when MDP is Known: Value Iteration

From Equation to Algorithm: If we know the market dynamics $P(s'|s,a)$ and rewards $R(s,a)$, we can turn the Bellman Optimality Equation into an **iterative update rule**.

Value Iteration for Q-Function

Initialize $Q_0(s,a)$ arbitrarily. For $k = 0, 1, 2, \ldots$ until convergence:

$$Q_{k+1}(s,a) = \underbrace{R(s,a)}_{\text{Immediate}} + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) \underbrace{\max_{a' \in \mathcal{A}} Q_k(s',a')}_{\text{Current Estimate of } V_k(s')}$$

Why use Dynamic Programming (DP)?

- **Convergence:** Under $\gamma < 1$, Q_k is guaranteed to converge to the unique Q^*.
- **Global Optimality:** Unlike local search, DP considers the entire state space.
- **Limitation in Finance:** Requires a "perfect model" of the market (P and R), which is rarely available in real-world

Value Iteration Algorithm

Objective: Find Q^* by repeatedly applying the Bellman Optimality operator.

1. **Initialize:** Set $Q_0(s, a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$.
2. **Iterate:** For $k = 0, 1, 2, \ldots$
   - For each state-action pair (s, a):

$$Q_{k+1}(s, a) \leftarrow R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \max_{a' \in \mathcal{A}} Q_k(s', a')$$

3. **Stop:** When $\max |Q_{k+1} - Q_k| < \epsilon$ (Convergence).
4. **Extract Policy:** $\pi^*(s) = \arg\max_a Q^*(s, a)$.

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Two Approaches to RL

RL methods can be broadly classified into two categories:

- **Model-Based RL**: First estimate an environment model, such as $\hat{P}(s'|s,a)$ and $\hat{R}(s,a)$.
  - **Process:** Experience $\rightarrow$ Learn Model $\rightarrow$ Plan / solve the learned MDP.
  - **Pros:** Often more sample efficient, since the learned model can be reused for simulation and planning.
  - **Cons:** Learning an accurate market model is notoriously difficult; model misspecification can lead to poor decisions.

- **Model-Free RL**: Learn value functions or policies directly from sampled experience, without explicitly learning a transition model for planning.
  - **Process:** Experience $\rightarrow$ Update Value / Policy directly.
  - **Pros:** Avoids explicit modeling of market dynamics, which can be difficult in complex financial environments.
  - **Cons:** Typically less sample efficient and may require large amounts of data.

Model-Free RL

Key Idea: The agent does not know the transition dynamics $P(s'|s, a)$. Instead, it learns directly from sampled experience through interaction with the environment.

Pros

- Does not require explicit modeling of the environment dynamics.
- Easier to apply when the environment is complex or difficult to model.

Cons

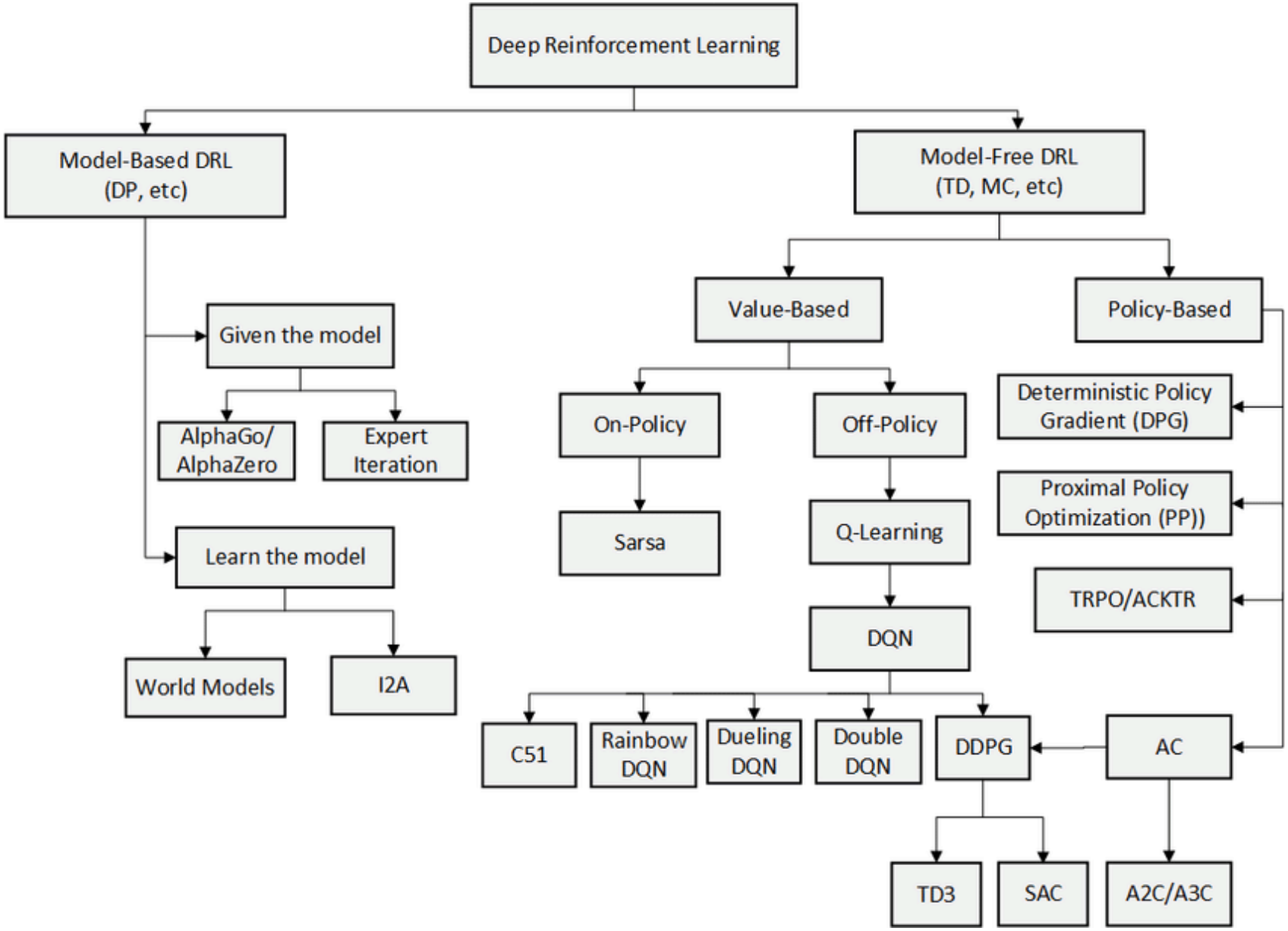
- Typically less sample efficient; requires more training data.
- Cannot perform explicit planning using a learned model.

Algorithms

Model-free RL algorithms can be broadly grouped into three categories:

- Value-Based Methods
  - Learn a value function (typically the Q-function).
  - The policy is obtained by choosing the action with the highest estimated value.
  - Examples: Q-Learning (off-policy), SARSA (on-policy)
- Policy-Based Methods
  - Directly parameterize and optimize the policy.
  - Typically use policy gradient methods.
  - Example: REINFORCE
- Hybrid Methods: Actor-Critic
  - Combine policy learning (actor) with value estimation (critic).
  - Examples: A2C, PPO, SAC

Algorithm List



Off-Policy Methods

Definition:

- Learn about the optimal policy π^* independent of the behavior policy.
- Updates use data that may come from a different (or older) policy.

Example: Q-Learning

$$Q^{(n+1)}(s,a) \leftarrow (1-\beta_n(s,a)) \underbrace{Q^{(n)}(s,a)}_{\text{current estimate}} + \beta_n(s,a) \underbrace{\left[r(s,a) + \gamma \max_{a'} Q^{(n)}\left(s', a'\right) \right]}_{\text{new estimate}}$$

- β is learning rate
- $\max_{a'} Q^{(n)}(s', a')$ is used even if the action taken in s' differs.

Key Benefit:

- Can learn from past experiences or data generated by a different policy.

Off-Policy Learning: The Q-Learning Update

Definition: Learning the optimal policy π^* while following a different *behavior policy*.

The Q-Learning Update Rule:

$$Q^{new}(s,a) \leftarrow (1-\beta)Q(s,a) + \beta \left[\underbrace{r}_{\text{Immediate}} + \gamma \max_{a'} \underbrace{Q(s',a')}_{\text{Best Future}} \right]$$

Alternative View: TD Error δ

$$Q(s,a) \leftarrow Q(s,a) + \beta \underbrace{\left[\overbrace{r + \gamma \max_{a'} Q(s',a')}^{\text{TD Target}} - Q(s,a) \right]}_{\text{TD Error (}\textit{Surprise!}\text{)}}$$

- **Exploration Independent:** We update Q based on the **best possible** next action (max), not necessarily the action we actually took in the sample.

Q-learning

ALGORITHM 2 *Q*-learning with samples from a policy π

Input: total number of iterations N; the policy π to be evaluated; rule to set learning rate $\beta_n \in (0, 1]$ $(0 \leq n \leq N-1)$
Initialize $Q^{(0)}(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$
Initialize s
for $n = 0, \cdots, N-1$ **do**
  Sample action a according to $\pi(s)$
  Observe r and s' after taking action a
  Update $Q^{(n+1)}$ according to (2.23) with sample (s, a, r, s')
  $s \leftarrow s'$
end for

On-Policy Learning

Definition: Updates are based on the **actual actions** taken by the current policy π, including its exploration (and potential mistakes).

The SARSA Update Rule:

$$Q(s,a) \leftarrow (1-\beta)Q(s,a) + \beta\Big[\underbrace{r}_{\text{Imm. Reward}} + \gamma \underbrace{Q(s',a')}_{\text{Value of } \textbf{actual} \text{ next action}}\Big]$$

- **S-A-R-S-A**: The name captures the full experience sequence: (s, a, r, s', a').
- **On-Policy**: It evaluates the value of the *current* policy π being followed, not a hypothetical optimal one.

SARSA vs. Q-Learning: The Strategic Trade-off

- **Q-Learning (**max**)**: **Optimistic**—assumes you will act perfectly in s'.
- **SARSA (a')**: **Realistic**—accounts for the fact that you are still "exploring" in s'.

SARSA

ALGORITHM 3 SARSA: On-policy TD learning

Input: total number of iterations N, the learning rate $\beta \in (0,1)^1$, and small parameter $\varepsilon > 0$.
Initialize $Q^{(0)}(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$
Initialize $s \in \mathcal{S}$
Initialize a: Choose a when in s using a policy derived from $Q^{(0)}$ (e.g, ε-greedy)
for $n = 0, 1, \cdots, N-1$ **do**
  Take action a, observe reward r and the next step s'
  Choose a' when in s' using a policy derived from $Q^{(n)}$ (e.g., ε-greedy)

$$Q^{(n+1)}(s,a) \leftarrow (1-\beta)\ \underbrace{Q^{(n)}(s,a)}_{\text{current estimate}} + \beta \underbrace{(r + \gamma Q^{(n)}(s',a'))}_{\text{new estimate}}$$

  $s \leftarrow s'$, $a \leftarrow a'$
end for

Action Selection: ε-Greedy Exploration

The Dilemma: How to choose a in s to balance Exploitation (current knowledge) and Exploration (seeking better paths)?

ε-Greedy Policy $\pi(a|s)$:

$$a = \begin{cases} \arg\max_{a'} Q(s, a') & \text{with probability } 1 - \varepsilon \quad \text{(Exploit)} \\ \text{random } a \in \mathcal{A} & \text{with probability } \varepsilon \quad \text{(Explore)} \end{cases}$$

The Difference in Updates

- **SARSA**: Updates $Q(s, a)$ using the **actual** next action a' chosen by this ε-greedy rule.
- **Q-Learning**: Updates $Q(s, a)$ using $\max_{a'} Q(s', a')$.

Policy Gradient: The REINFORCE Algorithm

The Shift: Directly parameterize the policy $\pi_\theta(a|s)$ (e.g., via a Neural Network) instead of maintaining a Q-table.

Objective Function: We want to maximize the expected total return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

Pros and Cons

+ **Continuous Action Spaces:** Can handle infinite action choices.
+ Directly optimizes the policy.
– **High Variance**: One lucky or unlucky trade can wildly swing the gradient.
– **Delayed Feedback**: Must wait until the end of the episode to update.

Policy Gradient Theorem (Sutton et al., 2000)

- The gradient of the expected return is given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\, G_t\right]$$

- Here, G_t is the return (cumulative discounted reward) from time t, i.e. $\sum_{s=t+1}^{M} \gamma^{s-t-1} r_s$.
- This "score function" $\nabla_\theta \log \pi_\theta(a|s)$ tells us how to adjust θ to increase the likelihood of taking actions that yield high rewards.
- **The Core Idea:**
  - Run an **entire episode** (e.g., a full trading day).
  - If the total return G_t is high, *increase* the probability of all actions taken.
  - If G_t is low, *decrease* their probability.

The REINFORCE Algorithm

Steps:

1. **Collect an Episode:** Run the current policy π_θ to generate a trajectory:
$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$$
2. **Compute Returns:** For each time step t, compute:
$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$
3. **Update Policy:** For each time step t, update parameters:
$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t$$
where α is the learning rate.

Actor–Critic Methods

- The approach consists of two main components:
  - **Actor:** Represents the policy $\pi_\theta(a|s)$, which selects actions.
  - **Critic:** Estimates the value function (e.g., $V^\pi(s)$ or $Q^\pi(s,a)$) to evaluate actions.

Actor:

- Parameterized policy $\pi_\theta(a|s)$
- Directly maps states to actions (or action probabilities)

Critic:

- Estimates the value function $V^\pi(s)$ or $Q^\pi(s,a)$
- Provides feedback on the expected long-term reward

Watch Actor-Critic

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Background and Motivation

- **Q-Learning Recap:**
  - Learns the optimal Q-function $Q^*(s, a)$ via:

$$Q^*(s, a) \leftarrow R(s, a) + \gamma \max_{a'} Q^*(s', a')$$

  - Traditional tabular methods become infeasible in high-dimensional state spaces.
- **Need for Function Approximation:**
  - Deep neural networks can approximate the Q-function.

What is DQN?

- **Deep Q-Networks (DQN)** use a deep neural network to approximate:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

- **Inputs:** Raw, high-dimensional state (e.g., game screen images).
- **Outputs:** Q-values for all possible actions.

DQN Algorithm Outline

1. Initialize Q-network with weights θ and target network with $\theta^- = \theta$.
2. For each episode:
   - Start from initial state s.
   - For each time step:
     - Select action a using an ϵ-greedy strategy.
     - Execute a, observe r and next state s'.
     - Store (s, a, r, s') in replay memory.
     - Sample a mini-batch from replay memory.
     - Compute target:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

     - Update Q-network by minimizing the loss:

$$L(\theta) = \mathbb{E}\left[(y - Q(s, a; \theta))^2\right]$$

   - Periodically update target network $\theta^- \leftarrow \theta$.

Why DQN is Important

- **Handles High-Dimensional Inputs:**
  - Learns directly from raw pixel data (e.g., Atari games).
- **Stability Enhancements:**
  - Experience replay and target networks reduce instability issues in learning.
- **Foundation for Extensions:**
  - Inspired numerous extensions (Double DQN, Dueling DQN, etc.) to improve performance.

Google DeepMind's Deep Q-learning playing Atari Breakout

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Optimal Execution

- **Early Approaches:**
  - Nevmyvaka et al. [2006]: Pioneered RL-based execution strategies.
- **Deep RL Breakthroughs:**
  - Ning et al. [2021]: Applied Deep Q-Networks (DQN) to handle high-dimensional market data.
- **Multi-Agent Simulations:**
  - Trading agent is situated in a simulated market alongside other agents (representing other traders or liquidity providers) to capture the interactive nature of order execution.
- **Institutional Deployment:**
  - RBC's Aiden: Uses deep RL for optimal execution.

Other Applications

- Beyond optimal execution, RL is applied in multiple areas of finance.
- Key domains include Hambly et al. [2023]:
  - Portfolio Management and Algorithmic Trading
  - Market Making and Risk Management
  - Option Pricing and Optimal Exercise Strategies
  - Credit Scoring and Fraud Detection

Portfolio Management & Algorithmic Trading

- **Portfolio Management:**
  - RL agents dynamically adjust asset allocations to optimize returns and control risk.
  - Strategies include dynamic rebalancing and risk-adjusted asset selection.
- **Algorithmic Trading:**
  - RL is used to develop trading strategies that determine entry and exit timing.
  - Applications span high-frequency trading, statistical arbitrage, and exploiting market inefficiencies.

Market Making & Risk Management

- **Market Making:**
  - RL agents set bid and ask quotes to provide liquidity while managing inventory risk.
  - The goal is to optimize the balance between profit from spreads and the cost of holding positions.
- **Risk Management and Hedging:**
  - RL techniques design dynamic hedging strategies.
  - The agent learns to adjust hedges based on market volatility and risk exposures.

Option Pricing, Exercise Strategies, & Fraud Detection

- **Option Pricing & Optimal Exercise:**
  - RL is applied to price American options and determine optimal exercise policies.
  - It handles the optimal stopping problem in a dynamic setting.
- **Credit Scoring & Fraud Detection:**
  - RL methods optimize decision policies for loan approvals and risk assessments.
  - They are also used to detect and adapt to emerging patterns in fraudulent activities.

Q&A

- Questions or thoughts about today's lecture?

Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3):437–503, 2023.